16-831: Introduction to Robot Learning
Course Project - CitySim RL

Nilay Pravin Marathe
Department of Mechanical Engineering
Carnegie Mellon University, United States
nmarathe@andrew.cmu.edu

Mugdha Meda
Department of Mechanical Engineering
Carnegie Mellon University, United States
mmeda@andrew.cmu.edu

1 Environment

Our goal is to build an environment that simulates a city's road architecture, including- multi-lane highways, merging roads, blocked roads, roundabouts and intersections. The ego vehicle that must learn to navigate this road network is surrounded with vehicles controlled by adaptable built-in behavior policies to emulate realistic driving scenarios. We use the open-source environments from the `HighwayEnv` repository [1] available on GitHub as a base to generate networks for our CitySim.

We started by stitching existing environments together, like the highway and merge. Each environment comes with its own lane building and coordinates that must be matched to have fruitful continuation of lanes, and the ego vehicle's state is handed off to the upcoming environment. A final custom stitched working environment: Highway-Adapter-Merge-Roundabout-Intersection was built, connecting a few existing environments with our own built lane networks to connect the drastically different environments that the ego vehcile must navigate through. Each 'NPC' vehicle is initialized with stochastic positions and velocities, interacting under realistic kinematic constraints. They are run on an Intelligent Driver Model (IDM) built into the `HighwayEnv` code base (`behavior.py`) and can be modified as required.

This composite environment enables a systematic evaluation of reinforcement learning policies under sequential, non-stationary traffic conditions. more closely resembling real-world driving scenarios than isolated single-task environments.

1.1 Action and Observation Space

The action space of the ego vehicle is discrete, consisting of high-level meta-actions for lane changing and speed control. The agent can choose from five actions: change lane to the left, or right, or remain in the current lane (idle), accelerate, or decelerate. These actions ensure focus is not on low-level control and allows the learning agent to focus on decision-making at the behavioral level.

The observation space is based on a kinematic representation of nearby vehicles. For each observed vehicle, the ego agent receives:

- Relative longitudinal distance
- Relative lateral distance
- Relative longitudinal velocity
- Relative lateral velocity
- Presence mask (binary indicator specifying whether a vehicle is within the observation range)

1.2 Reward Function

The goal for each environment varies, but there exists common goals for all environments. Trying to reach a good target speed while avoiding collisions, attempting to stay on the right, staying within

extremity lane boundaries, and preventing too many unnecessary lane changes is maintained across all segments.

However, there are also some objectives that belong to each particular environment. For example, in the Highway environment, the goal is to achieve high speeds. For Merge, the agent must make room for merging vehicles so that they can safely merge in traffic at a good enough speed. Roundabout asks for the ego vehicle to be able to get from entrance to exit among the many lanes and routes available, and Intersection, taking a successful turn and yielding to turn taking vehicles as well.

Each segment has different driving objectives, and our reward structure changes accordingly, with different weights to each objective.

All rewards are normalized between 0 and 1 for all sections of the stitched environment. The collision penalty is -1 and is highly weighted, neutralizing the highest possible reward.

Table 1: Reward Components by Environment Segment

Reward Component	**Highway**	**Merge**	**Roundabout**	**Intersection**
Collision Penalty (-1.0)	✓	✓	✓	✓
Lane Change Penalty (-0.05)		✓	✓	✓
Right Lane Reward (0.1)	✓	✓	✓	
High Speed Reward	✓(0.4)	✓(0.2)	✓(0.1)	✓(0.1)
Altruistic Merging Penalty (-0.5)		✓		
Progress Reward			✓(0.35)	✓(0.4)

2 Proposed Modifications

Changing the Environment and Effect on Policy:

2.1 Modification 1: Randomizing Environment Stitching

The current training environment is built by manually stitching the four environments together in the order- `HighwayEnv`, an AdapterEnv, `MergeEnv`, `RoundbaoutEnv`, and `IntersectionEnv` respectively. This environment however, is only for the initial attempt at environment stitching and training to understand learned behavior for different situations in a known environment (like the effect of changing NPC behavior, how NPCs behave with lane assignments in some environments, number of NPCs and spawn locations before drastic collisions). This will be extended to a city-like environment with wider possibilities of road architecture- random environments stitched together. The idea is for the ego vehicle to be capable of robust behavior across all the randomized environment segments.

In order to create the random environment, we will build wrapper functions that call the end segment coordinates of the previous environment, choose a new environment, hand off the end ego state, and build the right coordinate frame correctly. We hope to see the RL policy learn to accommodate for random orders of environments and not stick to a particular environment order.

2.2 Modification 2: Aggressive NPC Accommodation

To improve policy generalization across diverse traffic behaviors, we introduce a gradual increase in the aggressiveness of neighboring vehicles during training, and randomize what sort of behavior occurs for a spawned NPC. Rather than setting all NPCs to a fixed aggressiveness level, we vary each to follow different behaviors through a form of domain randomization. This adjustment exposes the ego vehicle to a spectrum of driving styles, ranging from conservative to increasingly assertive behaviors, allowing the learned agent to be more robust to variability in neighbor driver dynamics.

3 On-Policy RL, Off-Policy RL and Random Agent Analysis

- On-Policy RL: PPO (Proximal Policy Optimization)

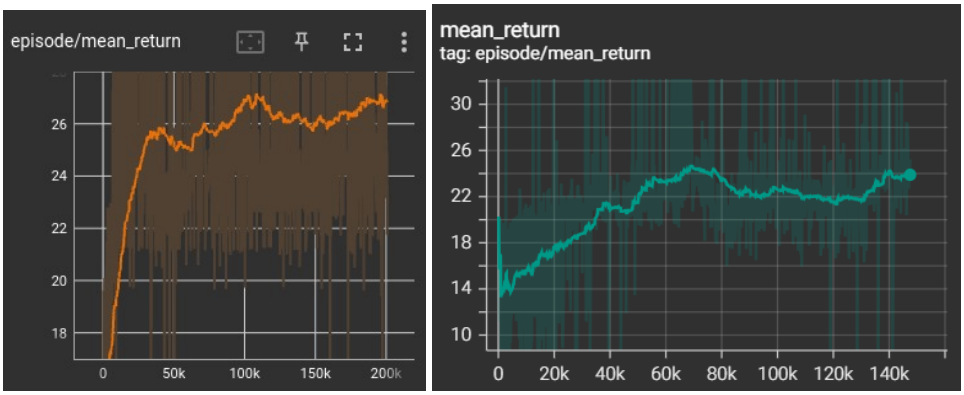
  PPO is an on-policy policy-gradient method, widely used with the `highway_env` benchmark due to its ability to handle stochastic environments and maintain stable updates through clipped objective functions. PPO directly optimizes the policy while limiting large policy updates, preventing performance collapse during training. It is particularly suitable for sequential decision-making tasks such as the stitched driving environments considered in this project.

- Off-Policy Q-Function Based RL: DQN (Deep Q-Network)

  DQN is a value-based off-policy algorithm that learns a Q-function mapping states to discrete action values, making it well-suited for environments with discrete action spaces and computationally efficient due to experience replay and target networks. Compared to algorithms like TD3 and SAC, which are designed for continuous control, DQN avoids the need to redesign our action space.

- Random Agent

  The action space is randomly sampled, changing lanes or speed at each discrete timestep irrespective of state or reward. The random agent serves as a baseline for evaluating the learning performance of PPO and DQN, helping determine how far the learned policies can outperform random behavior.



(a) Mean return v/s Env.Steps for PPO (b) Mean return v/s Env.Steps for DQN

Figure 1: Learning curves for PPO and DQN

3.1 Learning Curves

The rewards for each environment segment of the stitched environment are mapped between 0 and 1. Hence, an ideal agent taking the optimal actions at each timestep (where an average complete episode of our stitched environment consists of 40-50 timesteps) should result in a mean/cumulative return of around 40-50.

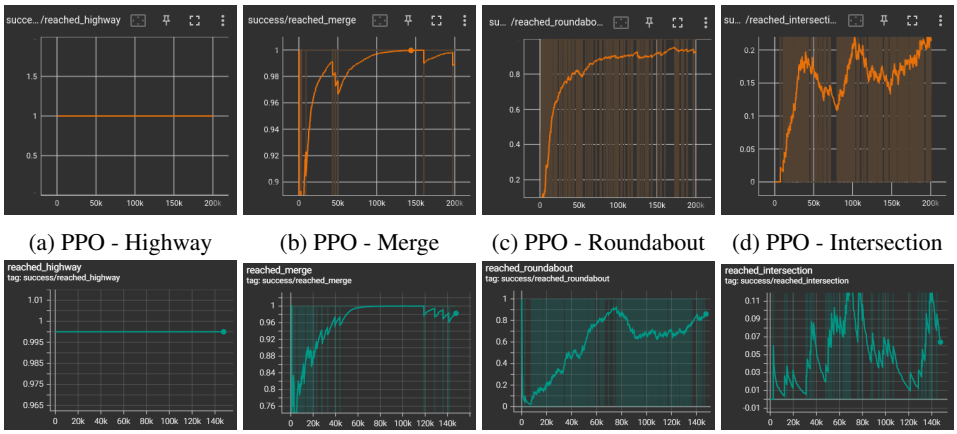
Our mean episodic return curves show a trend of plateauing around 25 for both DQN and PPO. This shows that the episode terminates (caused by a car crash) in the middle of our stitched environment-at the end of the Merge segment and the start of the Roundabout segment.

3.2 Success and Task Metrics

We log couple of different metrics to analyze performance. Given this is our first stitched environment, we see issues occurring due to chosen NPC behavior and spawning locations and interactions. We observe what prevents the policy from learning further as mentioned in the Learning Curves section.

To have a visual over whether the agent was able to learn an environment segment, we log ”Distance Covered” and ”Success in Reaching Environment Segment” for each stitched segment. We have the lengths of each environment’s lanes here and can hence infer how far we pass through in an episode through Distances. Successes provide binary values based on whether the agent/ego has ever set foot in the segment (hence why Highway-Success will always be 1 since we initialize our stitched environment with Highway).

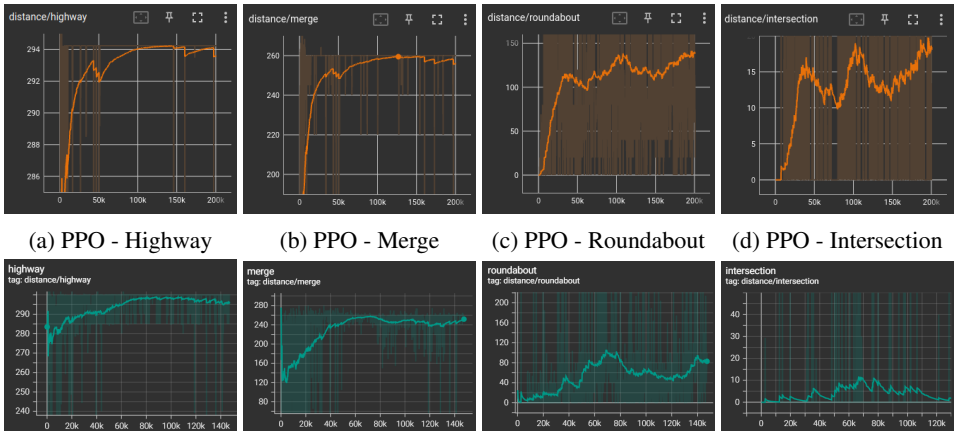
We see in Distances and Successes, that the policy is able to reach the end of the Highway and Merge segments, but fail at the start of the roundabout. We explain why we speculate so in the Failure Cases section and with visualizations on the website.



(a) PPO - Highway (b) PPO - Merge (c) PPO - Roundabout (d) PPO - Intersection

(e) DQN - Highway (f) DQN - Merge (g) DQN - Roundabout (h) DQN - Intersection

Figure 2: Success Metric for each environment segment



(a) PPO - Highway (b) PPO - Merge (c) PPO - Roundabout (d) PPO - Intersection

(e) DQN - Highway (f) DQN - Merge (g) DQN - Roundabout (h) DQN - Intersection

Figure 3: Distance Metric for each environment segment

3.3 Failure Cases

Observations behind failure of policy to achieve expected mean return per episode despite good hyperparameters (hyperparameters that function well for runs on individual environments, seen in their respective learning curves) provided to the RL policies:

1) Too many vehicles congress in short spatial intervals, particularly near the boundaries between consecutive road segments (e.g., highway–merge and merge–roundabout transitions). This results in local traffic bottlenecks with multiple 'NPC' vehicles trying to pass through at once. This often results in deadlocks and collisions.

2) Some vehicles entering the roundabout fail to exit within a reasonable time horizon and continue circulating in the roundabout indefinitely. This creates artificial deadlocks and obstacles increasing collision risk at the roundabout entrance, as incoming vehicles encounter continuously occupied lanes with limited gaps. This seems to a bug in the original code base, as certain vehicles always leave the roundabout and only a few vehicles always remain. This is perhaps a major reason as to why the average return plateaus around 25. Only a few cases are present where the ego vehicle actually crosses the roundabout and reaches the Intersection environment.

3) Vehicles are sometimes initialized at distances that are too small relative to their velocities and braking capabilities. Under these conditions, the vehicles following the Intelligent Driver Model (IDM) do not have sufficient time or space to safely adjust their acceleration or deceleration. This results in immediate near-collision scenarios. Frequent early collisions destabilize reinforcement learning by generating noisy rewards. Terminating episodes prematurely results in slow learning.

3.4 Next Steps

Solutions to issues observed in Failure Cases to be implemented, or implemented:

1. Slightly fewer NPCs will be spawned with better gaps between each other, and have larger lookahead. More importantly, NPC behavior must be re-adjusted to also have further slowing down and altruistic behavior at such bottlenecks.

2. We wish to implement a directed node set implementation for the vehicles that don't seem to find the exit node from the network graph of the stitched environment. While 90% of the vehicles do not face this issue, we can code a function that marks when a vehicle traverses the same nodes twice and force it follow a path leading to an exit.

3. In order to initialize the vehicles at appropriate distances, we take help of a normal distribution along the length of the highway lanes. If the distance between two cars was less than the minimum allowed distance, the difference was added such that the spawn position was shifted accordingly. This ensures sparse vehicle spawning removing the cases with early collisions.

Our website has the necessary plots and videos to visualize. [2].

4 Conclusions

We specified and would later study the effect of modifications to the environment on the ability of the learning model. We propose two key modifications. First, we will create a new environment as a combination of all the environments upon which we train. If training is appropriate, the ego vehicle should identify and adapt to the scenario it encounters. Second, we will increase the difficulty of the environment by making surrounding vehicles more aggressive by modifying the IDM parameters.

We will evaluate performance using mean episodic return plotted against total environment steps, along with video demonstrations. We expect the learned models to significantly improve driving safety and stability while maintaining efficient navigation.

References

[1] E. Leurent. An environment for autonomous driving decision-making. https://github.com/eleurent/highway-env, 2018.

[2] S26-16831-rlproject website. https://s26-16831-rlproject.github.io/, 2018.